



**UNIVERSIDADE FEDERAL DO MARANHÃO
CENTRO DE CIÊNCIAS EXATAS E TECNOLOGIA
BACHARELADO INTERDISCIPLINAR EM CIÊNCIA E TECNOLOGIA**

DISCIPLINA: LABORATÓRIO DE PROGRAMAÇÃO

**THALITA MUNIZ AMORIM
DEBORAH SOPHIA SOARES AGUIAR
SAMUEL CARVALHO DE MARIA
SAMIRA FARIAS MENDES**

**TRABALHO PRÁTICO - LABORATÓRIO DE PROGRAMAÇÃO
TRÊS PARADIGMAS, UM PROBLEMA
PROJETO ESCOLHIDO: SISTEMA ESPECIALISTA PARA DIAGNÓSTICO DE
FALHAS AUTOMOTIVAS**

**SÃO LUÍS - MA
2026**

TRABALHO PRÁTICO - LABORATÓRIO DE PROGRAMAÇÃO
TRÊS PARADIGMAS, UM PROBLEMA

PROJETO ESCOLHIDO: SISTEMA ESPECIALISTA PARA DIAGNÓSTICO DE
FALHAS AUTOMOTIVAS

Trabalho apresentado à disciplina de Laboratório de programação do Curso de Bacharelado em Interdisciplinar em ciência e tecnologia da Universidade Federal do Maranhão, como requisito para nota da 1, 2 e 3 avaliação da disciplina.

Professor(a):SOFIANE BEN EL HEDI LABIDI

SÃO LUÍS - MA
2026

1. Introdução.....	3
2. Objetivo.....	3
3. Descrição do Problema.....	3
4. Implementação em Python (Orientação a Objetos).....	3
Classes utilizadas.....	3
Exemplo de estrutura.....	3
Vantagens.....	3
Desvantagens.....	3
5. Implementação em Prolog (Programação Lógica).....	3
Vantagens.....	3
Desvantagens.....	3
6. Implementação em Lisp (Programação Funcional).....	3
Vantagens.....	3
Desvantagens.....	3
7. Comparação entre os Paradigmas.....	3
8. Facilidades Encontradas.....	3
Python.....	3
Prolog.....	3
Lisp.....	3
9. Dificuldades Encontradas.....	3
Python.....	3
Prolog.....	3
Lisp.....	3
10. Reflexão sobre Linguagens Multiparadigma.....	3
11. Conclusão.....	3
12. Referências.....	3

Introdução

A evolução da tecnologia da informação tem proporcionado o desenvolvimento de sistemas computacionais cada vez mais inteligentes, capazes de auxiliar profissionais na resolução de problemas complexos por meio da análise de dados, da aplicação de regras e da automatização de processos de tomada de decisão. Entre essas aplicações, destacam-se os sistemas especialistas, que representam um dos ramos mais tradicionais da Inteligência Artificial (IA). Esses sistemas têm como principal objetivo reproduzir o conhecimento e o raciocínio de especialistas humanos em um domínio específico, utilizando uma base de conhecimento composta por fatos e regras de inferência para oferecer soluções ou recomendações diante de determinadas situações.

Na área automotiva, a identificação correta de falhas mecânicas, elétricas e eletrônicas constitui uma etapa fundamental para garantir a segurança, o desempenho e a confiabilidade dos veículos. Entretanto, o processo de diagnóstico nem sempre é simples, uma vez que diferentes defeitos podem apresentar sintomas semelhantes, enquanto uma mesma falha pode ocasionar diversos sinais distintos. Essa característica exige que o profissional responsável possua experiência técnica, capacidade de interpretação e amplo conhecimento sobre o funcionamento dos diversos sistemas que compõem um automóvel. Além disso, com a crescente incorporação de componentes eletrônicos, sensores e módulos de controle nos veículos modernos, o diagnóstico tornou-se ainda mais complexo, aumentando a necessidade de ferramentas computacionais capazes de auxiliar mecânicos e técnicos durante a inspeção e a manutenção.

Nesse contexto, os sistemas especialistas surgem como uma alternativa eficiente para apoiar o processo de diagnóstico automotivo. A partir da entrada de sintomas observados pelo usuário, como dificuldade na partida, superaquecimento do motor, luzes de advertência acesas no painel, ruídos incomuns ou perda de potência, o sistema realiza uma análise baseada em regras previamente estabelecidas e apresenta o diagnóstico mais provável, além de sugerir os componentes que devem ser inspecionados ou substituídos. Embora esses sistemas não substituam completamente a experiência do profissional, eles contribuem significativamente para reduzir o tempo de diagnóstico, minimizar erros de interpretação e padronizar os procedimentos adotados durante a manutenção.

O desenvolvimento de um sistema especialista também representa uma excelente oportunidade para estudar diferentes paradigmas de programação, uma vez que o mesmo problema pode ser modelado sob perspectivas distintas, explorando características específicas de cada abordagem computacional. Enquanto algumas linguagens privilegiam a organização dos dados por meio de objetos, outras concentram seus esforços na representação lógica do conhecimento ou no processamento funcional de estruturas imutáveis. Dessa forma, a implementação de um mesmo sistema em diferentes paradigmas permite compreender não apenas as diferenças sintáticas entre as linguagens, mas também as diferentes formas de raciocinar e resolver problemas computacionais.

Neste trabalho foi desenvolvido um Sistema Especialista para Diagnóstico de Falhas Automotivas utilizando três paradigmas de programação distintos: programação orientada a objetos, programação lógica e programação funcional, representados, respectivamente, pelas linguagens Python, Prolog e Lisp. A proposta segue a atividade prática da disciplina de Laboratório de Programação, cujo objetivo consiste em implementar uma mesma solução computacional em três paradigmas diferentes, analisando suas particularidades, vantagens, limitações e aplicações práticas. O problema escolhido consiste em desenvolver um sistema capaz de receber uma lista de sintomas informados pelo usuário, relacioná-los com uma base de conhecimento previamente construída e inferir a falha mais provável do veículo, indicando ainda quais componentes devem ser verificados durante o processo de manutenção.

Na implementação orientada a objetos, buscou-se representar os diferentes elementos do domínio automotivo por meio de classes, objetos, atributos e métodos. Componentes como bateria, motor, alternador, velas de ignição e sistema de arrefecimento foram modelados como entidades independentes, permitindo uma organização modular do sistema e facilitando futuras expansões da base de conhecimento. Essa abordagem enfatiza conceitos como encapsulamento, herança e reutilização de código, tornando o desenvolvimento mais estruturado e favorecendo a manutenção do software.

Na versão desenvolvida em Prolog, o foco foi direcionado para a representação declarativa do conhecimento. Diferentemente das linguagens imperativas, em que o programador descreve passo a passo como o algoritmo deve ser executado, o Prolog permite especificar fatos e regras lógicas que representam o conhecimento do especialista. A partir das informações fornecidas pelo usuário, o mecanismo de inferência da linguagem realiza automaticamente o

processo de unificação e retrocesso (backtracking), identificando quais regras são satisfeitas e produzindo o diagnóstico correspondente. Essa característica faz da programação lógica uma das abordagens mais adequadas para sistemas especialistas, justamente por aproximar a implementação computacional do raciocínio humano utilizado durante a tomada de decisão.

Já na implementação funcional, utilizando a linguagem Lisp, o sistema foi desenvolvido com base no processamento de listas e na utilização de funções puras e recursivas. Os sintomas fornecidos pelo usuário são tratados como estruturas de dados imutáveis, sendo submetidos a sucessivas operações de filtragem e transformação até que reste apenas a hipótese diagnóstica mais compatível. Essa abordagem reduz efeitos colaterais durante a execução do programa, favorece a modularidade das funções e incentiva um estilo de programação baseado na composição funcional, amplamente utilizado em aplicações modernas que exigem maior confiabilidade e facilidade de paralelização.

Além do desenvolvimento do software, este trabalho busca realizar uma análise crítica sobre a influência do paradigma de programação na forma de modelar e resolver um mesmo problema computacional. Embora todas as implementações tenham como objetivo produzir o mesmo resultado funcional, cada paradigma oferece mecanismos distintos para representar informações, organizar o código e processar dados. Assim, pretende-se identificar quais aspectos foram facilitados ou dificultados em cada linguagem, avaliando qual delas apresentou maior adequação para o desenvolvimento de um sistema especialista voltado ao diagnóstico de falhas automotivas.

Por fim, espera-se que este estudo contribua para ampliar a compreensão sobre os diferentes paradigmas de programação e demonstre que a escolha da linguagem vai além de preferências pessoais ou questões de desempenho. A natureza do problema, a forma como o conhecimento é representado e os mecanismos necessários para sua resolução exercem influência direta na escolha da tecnologia mais apropriada. Dessa maneira, a experiência obtida com a implementação do mesmo sistema em Python, Prolog e Lisp permite compreender, na prática, as potencialidades e limitações de cada paradigma, bem como a importância das linguagens multiparadigma no desenvolvimento de soluções computacionais modernas.

Objetivo

O presente trabalho tem como objetivo desenvolver um Sistema Especialista para Diagnóstico de Falhas Automotivas capaz de auxiliar na identificação das causas mais prováveis de defeitos em veículos a partir da análise de sintomas informados pelo usuário. O sistema foi concebido para simular o raciocínio utilizado por um mecânico experiente durante o processo de diagnóstico, relacionando diferentes sintomas a possíveis falhas mecânicas, elétricas e eletrônicas, além de fornecer recomendações sobre os componentes que devem ser inspecionados ou substituídos. A proposta busca demonstrar como técnicas de programação podem ser empregadas para representar conhecimento especializado e apoiar a tomada de decisões em situações que normalmente exigem elevado nível de experiência técnica.

Além da construção do sistema especialista, este trabalho tem como objetivo principal comparar três paradigmas distintos de programação por meio da implementação da mesma solução computacional nas linguagens Python, Prolog e Lisp, representando, respectivamente, os paradigmas orientado a objetos, lógico e funcional. A utilização de um único problema para os três paradigmas permite realizar uma comparação justa entre as diferentes abordagens, evidenciando como cada linguagem organiza os dados, representa o conhecimento, processa as informações e implementa o mecanismo responsável pela geração do diagnóstico. Dessa forma, pretende-se compreender como o paradigma influencia diretamente o processo de desenvolvimento de software e quais características tornam determinada abordagem mais adequada para problemas específicos.

Na implementação orientada a objetos, pretende-se modelar o domínio automotivo utilizando classes que representem os principais elementos envolvidos no diagnóstico, como veículos, componentes, falhas, sintomas e recomendações de manutenção. Essa modelagem busca explorar conceitos fundamentais da programação orientada a objetos, como encapsulamento, abstração, herança e reutilização de código, permitindo que o sistema apresente uma estrutura organizada, modular e de fácil manutenção. Espera-se demonstrar como esse paradigma facilita a representação de sistemas compostos por diferentes entidades inter-relacionadas, tornando o código mais intuitivo e preparado para futuras expansões da base de conhecimento.

Na implementação utilizando Prolog, o objetivo consiste em representar o conhecimento do especialista por meio de fatos e regras lógicas, permitindo que o próprio mecanismo de inferência da linguagem seja responsável por identificar o diagnóstico mais provável. Nessa abordagem, procura-se explorar características típicas da programação lógica, como unificação, resolução automática de consultas e retrocesso (backtracking), evidenciando como esse paradigma se adapta naturalmente à construção de sistemas especialistas baseados em regras do tipo "SE... ENTÃO...". Espera-se verificar de que maneira a programação declarativa reduz a complexidade dos algoritmos de decisão, uma vez que o programador descreve apenas o conhecimento do domínio, enquanto a linguagem se encarrega do processo de inferência.

Na implementação funcional, utilizando a linguagem Lisp, pretende-se desenvolver o sistema com base em listas imutáveis e funções puras, explorando conceitos como recursão, composição funcional e ausência de efeitos colaterais. O objetivo é demonstrar como operações sucessivas de filtragem e transformação podem ser utilizadas para reduzir o conjunto inicial de hipóteses até que reste apenas a falha mais compatível com os sintomas informados. Essa implementação busca evidenciar as vantagens da programação funcional na construção de algoritmos modulares, previsíveis e de fácil reutilização, além de destacar sua contribuição para aplicações que exigem maior confiabilidade e facilidade de paralelização.

Outro objetivo importante deste trabalho consiste em analisar criticamente as vantagens e limitações de cada paradigma durante o processo de desenvolvimento. Para isso, serão observados aspectos como facilidade de implementação, clareza do código, organização estrutural, reutilização de componentes, facilidade para manutenção, capacidade de expansão do sistema, desempenho da solução e simplicidade para representar o conhecimento necessário ao diagnóstico automotivo. A comparação não se limitará aos resultados obtidos pelo software, mas também considerará a experiência prática de desenvolvimento, identificando quais tarefas foram mais simples ou mais complexas em cada linguagem e como o paradigma influenciou as decisões de projeto adotadas durante a implementação.

Também faz parte dos objetivos compreender como diferentes paradigmas podem ser complementares em vez de concorrentes. Embora cada linguagem possua características próprias, espera-se demonstrar que determinadas partes do problema podem ser resolvidas de maneira mais eficiente utilizando abordagens distintas. A modelagem das entidades do

sistema, por exemplo, tende a ser mais intuitiva na programação orientada a objetos, enquanto o mecanismo de inferência apresenta melhor desempenho conceitual na programação lógica. Já o processamento de listas e o tratamento das informações podem ser realizados de forma elegante e concisa por meio da programação funcional. Essa análise pretende evidenciar que a escolha da linguagem mais adequada depende diretamente da natureza do problema a ser resolvido.

Além dos aspectos técnicos relacionados ao desenvolvimento do software, este trabalho busca ampliar a compreensão sobre o conceito de linguagens multiparadigma e sua importância no contexto atual da engenharia de software. Muitas linguagens modernas incorporam características de diferentes paradigmas, permitindo ao desenvolvedor selecionar a abordagem mais apropriada para cada situação. Assim, outro objetivo deste estudo é refletir sobre os benefícios e desafios da integração entre programação orientada a objetos, lógica e funcional em uma única linguagem, discutindo como essa combinação pode aumentar a produtividade, melhorar a qualidade do código e tornar os sistemas mais flexíveis e adaptáveis às diferentes necessidades encontradas no desenvolvimento de aplicações reais. Essa reflexão está alinhada à proposta da disciplina, que incentiva a comparação entre paradigmas e a discussão sobre linguagens multiparadigma como Scala, Kotlin, Rust e Clojure.

Por fim, espera-se que o desenvolvimento deste projeto contribua para consolidar os conhecimentos adquiridos ao longo da disciplina de Laboratório de Programação, proporcionando uma experiência prática na utilização de três linguagens com características bastante distintas. Ao implementar um mesmo sistema em Python, Prolog e Lisp, pretende-se não apenas compreender as diferenças sintáticas entre essas linguagens, mas também desenvolver uma visão crítica sobre a forma como cada paradigma influencia a modelagem de problemas, a organização do código e o processo de construção de soluções computacionais. Dessa maneira, o trabalho busca demonstrar que a escolha de um paradigma de programação deve considerar não apenas aspectos técnicos, mas também a natureza do problema, a facilidade de manutenção do software e os objetivos específicos da aplicação desenvolvida.

Descrição do Problema

O sistema considera sintomas comuns encontrados em veículos, tais como:

- motor não liga;
- bateria descarregada;
- luz da bateria acesa;
- ruído metálico;
- superaquecimento;
- fumaça branca;
- consumo elevado de combustível;
- dificuldade na partida;
- perda de potência;
- falhas na ignição.

Com base nessas informações, o programa deve identificar problemas como:

- bateria descarregada;
- alternador defeituoso;
- bomba de combustível com defeito;
- vela de ignição desgastada;
- junta do cabeçote danificada;
- falha no sistema de arrefecimento;
- desgaste interno do motor;
- problemas na injeção eletrônica.

Após o diagnóstico, o sistema recomenda os componentes que devem ser inspecionados ou substituídos.

Implementação em Python (Orientação a Objetos)

Na implementação orientada a objetos, o problema foi modelado utilizando classes que representam os elementos do domínio.

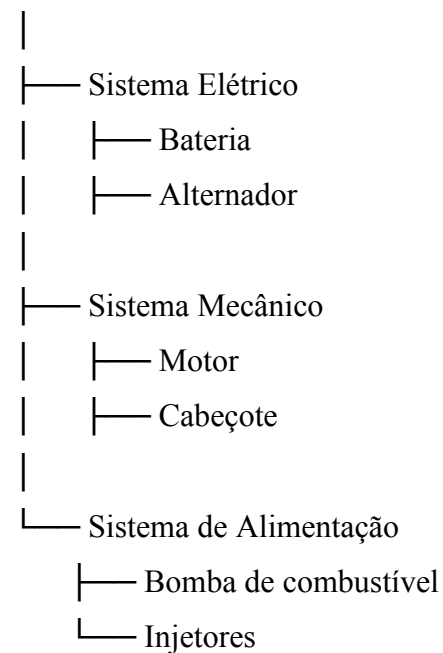
Classes utilizadas

- Carro
- Componente
- Falha
- Diagnóstico
- Sistema Especialista

Cada objeto possui atributos próprios, como nome do componente, sintomas associados, possíveis falhas e recomendações de manutenção.

Exemplo de estrutura

Carro



O algoritmo percorre a lista de sintomas fornecida pelo usuário, compara com as regras cadastradas e retorna o diagnóstico correspondente.

Vantagens

- código organizado;
- fácil manutenção;
- reutilização de classes;
- facilidade para adicionar novos componentes.

Desvantagens

- maior quantidade de código;
- necessidade de criar diversas classes para representar o domínio.

Implementação em Prolog (Programação Lógica)

Na implementação em Prolog, o conhecimento do especialista é representado por fatos e regras.

Exemplo:

```
sintoma(motor_ao_liga).  
sintoma(luz_bateria).
```

```
falha(bateria) :-  
    sintoma(motor_ao_liga),  
    sintoma(luz_bateria).
```

Quando o usuário informa os sintomas, o Prolog utiliza unificação e inferência lógica para encontrar a regra que melhor explica a situação.

Vantagens

- representação natural de conhecimento;
- código pequeno;
- inferência automática;
- fácil inclusão de novas regras.

Desvantagens

- menor familiaridade para programadores iniciantes;
- pouca flexibilidade para interfaces gráficas;
- depuração mais difícil.

Implementação em Lisp (Programação Funcional)

Em Lisp, os sintomas são tratados como listas imutáveis.

```
("motor_nao_liga"  
"luz_bateria"  
"partida_fraca")
```

As funções recebem essas listas e aplicam filtros sucessivos até restarem apenas as hipóteses compatíveis.

Fluxo:

Sintomas

↓

Filtragem

↓

Eliminação das hipóteses



Falha provável



Recomendação

Vantagens

- funções pequenas;
- código modular;
- ausência de efeitos colaterais;
- facilidade para reutilização.

Desvantagens

- recursão pode ser difícil para iniciantes;
- sintaxe pouco intuitiva;
- curva de aprendizagem maior.

Comparação entre os Paradigmas

Característica	Python	Prolog	Lisp
Organização	Excelente	Boa	Boa
Facilidade de leitura	Muito alta	Média	Média
Inferência automática	Não	Sim	Não
Facilidade para adicionar regras	Média	Excelente	Boa
Reutilização	Excelente	Média	Excelente
Interface gráfica	Excelente	Limitada	Limitada
Curva de aprendizagem	Baixa	Alta	Alta

Facilidades Encontradas

Python

A organização em classes facilitou representar os diferentes sistemas do veículo e manter o código modular.

Prolog

Foi a linguagem mais adequada para representar conhecimento especializado. As regras do tipo "SE... ENTÃO..." refletem diretamente a forma como um mecânico realiza o diagnóstico.

Lisp

A manipulação de listas tornou simples o processamento dos sintomas e a eliminação progressiva das hipóteses.

Dificuldades Encontradas

Durante o desenvolvimento do Sistema Especialista para Diagnóstico de Falhas Automotivas, foram identificados desafios específicos em cada paradigma de programação. Essas dificuldades não estiveram relacionadas apenas à sintaxe das linguagens, mas principalmente à maneira como cada paradigma exige que o problema seja modelado e resolvido.

Python (Programação Orientada a Objetos)

Na implementação em Python, a principal dificuldade foi estruturar corretamente o sistema utilizando os princípios da orientação a objetos. Foi necessário definir classes, atributos, métodos e relacionamentos entre os diferentes componentes do domínio automotivo, como veículos, sistemas mecânicos, componentes e falhas. Embora essa organização proporcione um código mais modular e reutilizável, ela também aumenta a complexidade inicial do projeto, exigindo maior planejamento da arquitetura do software.

Outro desafio consistiu em implementar o mecanismo de diagnóstico. Como Python não possui um sistema de inferência lógica nativo, foi necessário desenvolver manualmente toda a lógica de comparação entre sintomas e possíveis falhas, utilizando estruturas condicionais, listas e dicionários. À medida que a base de conhecimento cresce, essa abordagem tende a tornar o código mais extenso e complexo, dificultando a manutenção e a inclusão de novas regras de diagnóstico.

Prolog (Programação Lógica)

O Prolog apresentou a maior mudança de paradigma em relação às linguagens tradicionalmente utilizadas durante a graduação. A principal dificuldade foi compreender o modelo declarativo de programação, no qual o desenvolvedor descreve fatos e regras em vez de especificar passo a passo como o algoritmo deve ser executado.

Além disso, foi necessário compreender o funcionamento dos mecanismos internos da linguagem, como unificação, retrocesso (*backtracking*) e resolução automática de consultas. Durante a implementação, pequenas inconsistências na definição das regras podiam impedir que o sistema encontrasse uma solução ou gerassem diagnósticos incorretos, tornando o processo de depuração mais trabalhoso.

Outro aspecto observado foi que, embora o Prolog seja extremamente eficiente para representar conhecimento baseado em regras, sua integração com interfaces gráficas, bancos de dados e outras bibliotecas é mais limitada quando comparada a linguagens amplamente utilizadas no desenvolvimento de aplicações, como Python.

Lisp (Programação Funcional)

Na implementação em Lisp, a principal dificuldade esteve relacionada à adaptação ao paradigma funcional. Diferentemente das linguagens imperativas, em que o fluxo de execução é baseado na alteração de variáveis ao longo do programa, o Lisp incentiva o uso de funções puras, estruturas imutáveis e recursão para resolver os problemas.

A sintaxe baseada em expressões aninhadas e o grande número de parênteses também exigiram um período de adaptação, especialmente durante a leitura e manutenção do código. Inicialmente, a identificação de erros tornou-se mais difícil devido à complexidade visual das expressões.

Outro desafio foi desenvolver o algoritmo de diagnóstico utilizando exclusivamente operações sobre listas e chamadas recursivas. Embora essa abordagem resulte em funções mais modulares e reutilizáveis, ela exige uma forma de raciocínio diferente da programação imperativa, tornando o processo de desenvolvimento mais complexo para programadores iniciantes.

Comparação Geral das Dificuldades

De forma geral, observou-se que as dificuldades encontradas não estavam apenas relacionadas ao aprendizado das linguagens, mas principalmente à mudança na forma de pensar a solução do problema.

A programação orientada a objetos exigiu maior esforço na modelagem e organização das entidades do sistema; a programação lógica demandou compreender o funcionamento da inferência automática e da representação declarativa do conhecimento; enquanto a programação funcional exigiu a adaptação ao uso de funções puras, recursão e estruturas imutáveis.

Essas diferenças evidenciam que cada paradigma possui características próprias que influenciam diretamente a forma de desenvolver uma aplicação. Apesar dos desafios iniciais, a implementação do mesmo sistema nos três paradigmas proporcionou uma compreensão mais ampla sobre diferentes estratégias de resolução de problemas e demonstrou que a escolha da abordagem mais adequada depende das características da aplicação e dos objetivos do projeto.

Reflexão sobre Linguagens Multiparadigma

Durante o desenvolvimento foi possível perceber que nenhum paradigma resolve todos os problemas da melhor maneira.

O paradigma orientado a objetos oferece excelente organização do sistema, sendo ideal para representar entidades complexas e facilitar a manutenção do código. Por outro lado, a programação lógica mostrou-se superior para sistemas especialistas, pois permite representar conhecimento por meio de regras de inferência, eliminando a necessidade de implementar manualmente diversos algoritmos de decisão. Já a programação funcional destacou-se pelo processamento eficiente de listas de sintomas utilizando funções puras e estruturas imutáveis.

Essas características demonstram a importância das linguagens multiparadigma, que combinam diferentes estilos de programação em uma única linguagem. Entre os exemplos apresentados no material de apoio estão **Scala**, que integra programação orientada a objetos e funcional na plataforma JVM, e **Kotlin**, que combina paradigmas orientado a objetos, funcional e imperativo, oferecendo maior flexibilidade ao desenvolvedor.

A adoção de linguagens multiparadigma permite selecionar a abordagem mais adequada para cada parte do sistema, aumentando a produtividade, a qualidade do código e a facilidade de manutenção.

Conclusão

O desenvolvimento do Sistema Especialista para Diagnóstico de Falhas Automotivas permitiu comparar, na prática, três paradigmas distintos de programação aplicados ao mesmo problema.

Embora todos tenham sido capazes de produzir o mesmo resultado funcional, cada paradigma apresentou vantagens específicas. O Python destacou-se pela organização e facilidade de manutenção; o Prolog demonstrou ser a solução mais adequada para sistemas baseados em regras e inferência; enquanto o Lisp mostrou grande eficiência no processamento funcional dos sintomas.

A experiência evidenciou que a escolha da linguagem influencia diretamente a forma de modelar e resolver um problema computacional. Também reforçou a importância das linguagens multiparadigma, que reúnem características complementares e permitem ao desenvolvedor utilizar a abordagem mais apropriada para cada contexto.

Referências

- RUSSELL, Stuart; NORVIG, Peter. *Artificial Intelligence: A Modern Approach*. 4. ed. Pearson, 2021.
- BRATKO, Ivan. *Prolog Programming for Artificial Intelligence*. 4. ed. Pearson, 2011.
- LUTZ, Mark. *Learning Python*. 5. ed. O'Reilly, 2013.
- GRAHAM, Paul. *ANSI Common Lisp*. Prentice Hall, 1996.
- Material da disciplina **Laboratório de Programação – Trabalho Prático: Três Paradigmas, Um Problema**, UFMA, 2026.